

## Testing Agile

### Manifiesto

- Pruebas durante el desarrollo en vez de pruebas al final.
- Prevenir defectos en vez de encontrarlos.
- Entender lo que se está probando en vez de verificar la funcionalidad.
- Construir un mejor sistema en vez de intentar romperlo.
- Responsabilidad de todo el equipo por la calidad en vez de solo el tester.

### Principios

- Testing no es una fase.
- Todos hacen testing.
- Reducir la latencia del feedback.
- Pruebas representan expectativas.
- Reducir la sobrecarga de documentación de las pruebas.
- Desarrollo conducido por pruebas.

## Testing de Software

### Definición

Proceso destructivo con el objetivo de encontrar defectos en el código cuya presencia se asume. Es exitoso si encuentra defectos.

No asegura la calidad del producto

### Conceptos importantes

- Error: Se encuentra en la misma etapa en la que se produce.
- Defecto: Se encuentra en una etapa posterior a la que se produce. El testing encuentra defectos, no errores.

No todos los defectos tienen la misma severidad. La severidad tiene que ver con cuánto me imposibilita hacer cierto defecto. Puede ser bloqueante, crítico, mayor, menor o cosmético. La severidad no determina en qué orden se deben tratar. Esto se determina según la prioridad que se le da (según impacto del defecto en el contexto del negocio).

### Niveles de prueba

- Pruebas Unitarias:  
Ejecutado por desarrollador.  
Se prueban componentes individuales
- Pruebas de Integración:  
Se prueban componentes interactuando entre sí.  
La estrategia de integración incremental.
- Pruebas de Sistema:  
Mira el Sistema como un todo.  
Se realiza en un entorno que imita el entorno de producción. Comienza a tener en cuenta requerimientos funcionales y no funcionales.
- Pruebas de Aceptación:  
Son las pruebas de aceptación que figuran en las US.  
El foco es corroborar que el sistema hace lo que se pidió.

### Ambientes

- Desarrollo
- Prueba
- Pre-Producción
- Producción

## Proceso de Prueba

### Caso de prueba:

Set de condiciones o variables bajo las cuales un tester determinará si el software está funcionando correctamente o no.

Una buena definición de casos de prueba nos ayuda a reproducir defectos.

El objetivo es alcanzar la mayor cobertura posible con la menor cantidad de casos de prueba.

Tiene valores concretos, no genéricos.

Es la unidad de la actividad de la prueba

Está compuesto de 3 partes:

- Objetivo
- Datos de entrada y de ambiente
- Comportamiento esperado

### Condiciones de prueba:

Reacción esperada de un sistema frente a un estímulo particular, este estímulo está constituido por las distintas entradas. Una condición de prueba debe ser probada por al menos un caso de prueba.

## Estrategias de Prueba

Caja blanca: Definir casos de prueba teniendo en cuenta que haya cobertura de la mayor parte del código.

Se basan en el análisis de la estructura interna del software.

Caja negra: Implica ejecutar el software y definir el resultado obtenido que se debe comparar con los esperados.

- Basado en especificaciones
  - Partición de equivalencias
  - Análisis de valores límites
- Basado en la experiencia
  - Adivinanza de defectos
  - Testing exploratorio

### Ciclo de pruebas

Abarca la ejecución de la totalidad de los casos de prueba establecidos aplicados a una misma versión del sistema a probar.

### Test de Regresión

Se vuelven a ejecutar las pruebas una vez que se corrigen los defectos encontrados.

## Proceso de Pruebas

1. Planificación: Identificar y especificar casos de prueba, métodos, cobertura
  - a. Identificación
  - b. Especificación
2. Ejecución
3. Análisis de Fallas
4. Fin de pruebas

Validación: El correcto

Verificación: Correctamente

### Criterios de Aceptación

Se usa para resolver el problema de determinar cuándo una determinada fase de testing ha sido completada.

Puede ser definido en términos de costos, % de test corridos sin fallas, severidad de los defectos encontrados.

### Métodos de Prueba

#### Tipos de Prueba

Testing funcional y testing no funcional

## Filosofía Lean y Kanban

### Filosofía Lean

Principios:

- Eliminar desperdicios: Stock sobrante, defectos, burocrasia.
- Aplicar aprendizaje
- Embeber la integridad conceptual: Pensar desde el primer momento en el producto como un todo.
- Definir compromisos: Solo definir lo necesario y a último momento para disminuir desperdicios.
- Dar poder al equipo
- Ver el todo
- Entregar lo antes posible

Dentro de los desperdicios, hay desperdicios necesarios y desperdicios puros. Los necesarios no los puedo eliminar aún, los voy mejorando de a poco.

Clases de desperdicio:

- Talento no utilizado.
- Inventario: Trabajo parcialmente terminado.
- Movimientos: Cambio de contextos.
- Espera.
- Transporte: Cambio de tareas.
- Defectos.
- Sobreproducción: Características extra.
- Sobre-procesamiento: Procesos extra.

### Kanban

*Método* para definir, gestionar y mejorar servicios (procesos, no proyectos) que entregan trabajo del conocimiento, tales como servicios profesionales, trabajos o actividades en las que interviene la creatividad y el diseño tanto de productos de software como físicos.

Valores

- Transparencia
- Equilibrio
- Colaboración
- Foco en el cliente
- Flujo
- Liderazgo
- Entendimiento
- Acuerdo
- Respeto

Principios

- Gestión de cambios
  - Comenzar con lo que haces ahora
  - Acordar la búsqueda de la mejora a través del cambio evolutivo
  - Fomentar actos de liderazgo a todos los niveles
- Entrega de Servicios
  - Comprender y enfocarse en cumplir las necesidades y expectativas del cliente
  - Gestionar el trabajo; dejar que los trabajadores se auto organicen en torno a él
  - Revisar periódicamente la red de servicios y sus políticas para mejorar los resultados entregados

#### Prácticas generales

- Visualizar: Muestra el trabajo y su flujo. Visualiza los riesgos. El equipo sabe que es lo que se está haciendo.
- Limitar el trabajo en curso: Empezar y terminar cosas. No hacer muchas cosas a la vez.
- Gestionar el flujo: El flujo es el movimiento del trabajo. Los trabajos son autoasignados (pull, no push)
- Hacer las políticas explícitas: Determinar criterios para asignación de trabajo, límites de trabajo en curso, clases de servicios.
- Establecer ciclos de retroalimentación: Fomenta colaboración, aprendizaje y mejoras.
- Mejorar colaborativamente, evolucionar experimentalmente: Cambios impulsados por hipótesis.

#### Limitar el trabajo en progreso:

Los sistemas eficaces son los que se centran más en el flujo de trabajo que en tener ocupados a los trabajadores. Si los recursos están 100% ocupados, no hay holgura ni capacidad para adaptarse.

## Métricas

#### Aproach minimalísta de las métricas

- Lead time: Tiempo entre que el cliente pide algo y sale del tablero.
- Cycle Time: Tiempo desde que empieza a trabajar una actividad en el tablero hasta que sale.
- Touch time: Tiempo efectivo que se trabajó en una actividad. Su diferencia con el Cycle Time nos muestra el cuello de botella.

#### Las métricas están orientadas a servicio

- Definir acuerdo de nivel de servicio con cliente
- Capacidad del nivel de servicio al que el sistema puede entregar
- Expectativa de nivel de servicio que los clientes esperan
- Umbral de adecuación del servicio el nivel por debajo del cual este es inaceptable para el cliente